AcademiX

Plataforma Educativa — Arquitectura de Microservicios

Documentación Técnica y Guía de Despliegue

Elkin Mauricio Idarraga Valencia

Ingeniería de Software II • 2026

Índice

Índice **2**
1. Descripción General del Sistema **4**
1.1 Microservicios del Sistema 4
1.2 Diagrama de Arquitectura 4
1.3 Flujo de una Petición 5
2. Autenticación y Seguridad **6**
2.1 Autenticación de Clientes — Laravel Sanctum 6
2.2 Token Interno entre Microservicios 6
2.3 Validaciones del Gateway 6
3. API Gateway + Usuarios (Laravel 11 — Puerto 8000) **7**
3.1 Instalación de Dependencias 7
3.2 Variables de Entorno (.env) 7
3.3 Tabla: users 8
3.4 Endpoints 8
Autenticación 8
Cursos (proxy → courses_service) 8
Módulos (proxy → content_service) 8
Lecciones (proxy → content_service) 9
Inscripciones (proxy → enrollments_service) 9
Evaluaciones (proxy → evaluations_service) 9
4. Courses Service (Flask — Puerto 5000) **10**
4.1 Instalación de Dependencias 10
4.2 Variables de Entorno (.env) 10
4.3 Tabla: courses 10
4.4 Endpoints 11
5. Content Service (Express/Node.js — Puerto 3000) **11**
5.1 Instalación de Dependencias 11
5.2 Variables de Entorno (.env) 11
5.3 Colecciones MongoDB 11
modules 11
lessons 12
5.4 Endpoints 12
Módulos 12
Lecciones 12
6. Enrollments Service (Django — Puerto 8080) **13**
6.1 Instalación de Dependencias 13
6.2 Variables de Entorno (.env) 13
6.3 Tabla: enrollments 13
6.4 Endpoints 13
7. Evaluations Service (Flask — Puerto 5001) **14**
7.1 Instalación de Dependencias 14
7.2 Variables de Entorno (.env) 14
7.3 Tablas 14

exams 14
questions 15
submissions 15
7.4 Endpoints 15
Exámenes 15
Preguntas 15
Submissions 16
8. Diagrama de arquitectura 17
9. Estructura del Repositorio 18
10. Pruebas de rendimiento 19
10.1 Resultados – Microservicio Courses 19
10.2 Resultados – Microservicio Enrollments 24
10.3 Resultados – Microservicio Content 27
10.4 Resultados – Microservicio Evaluations 30
11. Guía de Despliegue Completo 35
Paso 1 — Clonar el repositorio 35
Paso 2 — Configurar bases de datos 35
Paso 3 — Levantar cada servicio 35
Gateway (Laravel) 35
Courses Service (Flask) 35
Content Service (Express) 36
Enrollments Service (Django) 36
Evaluations Service (Flask) 36
Paso 4 — Verificar que todos los servicios estén activos 36

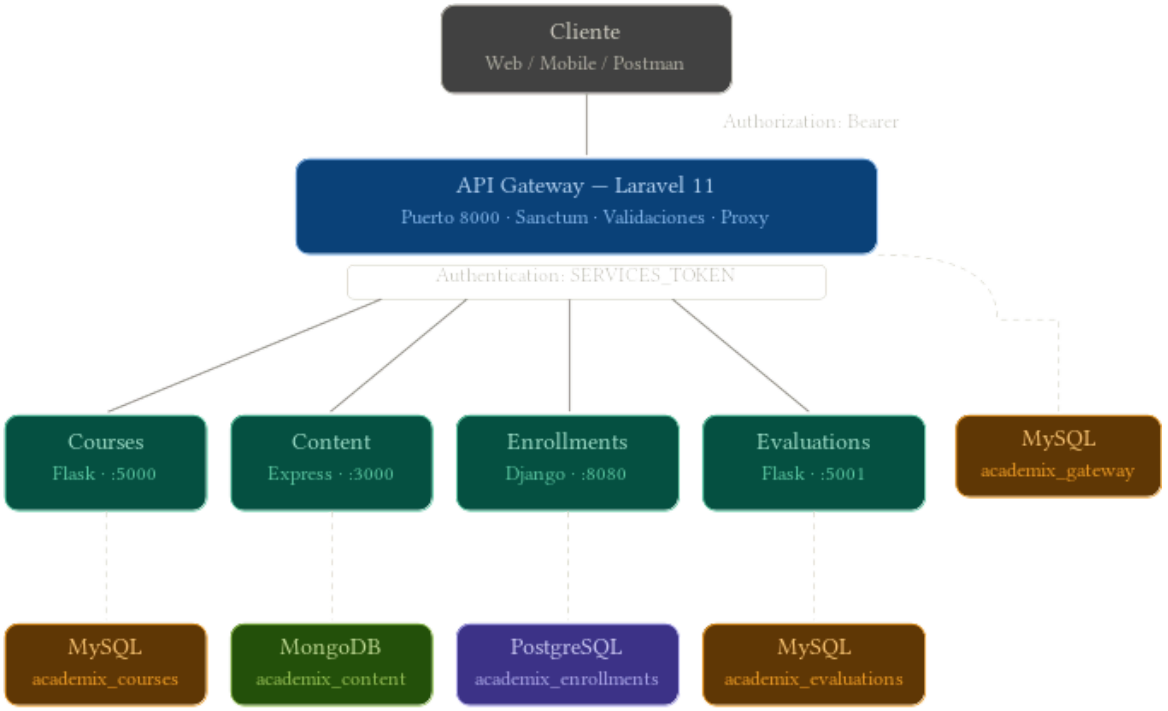
1. Descripción General del Sistema

AcademiX es una plataforma educativa tipo Moodle/Udemy construida sobre una arquitectura de microservicios. Cada servicio es completamente independiente, posee su propia base de datos y se comunica exclusivamente a través del API Gateway. Ningún cliente externo puede acceder directamente a los microservicios.

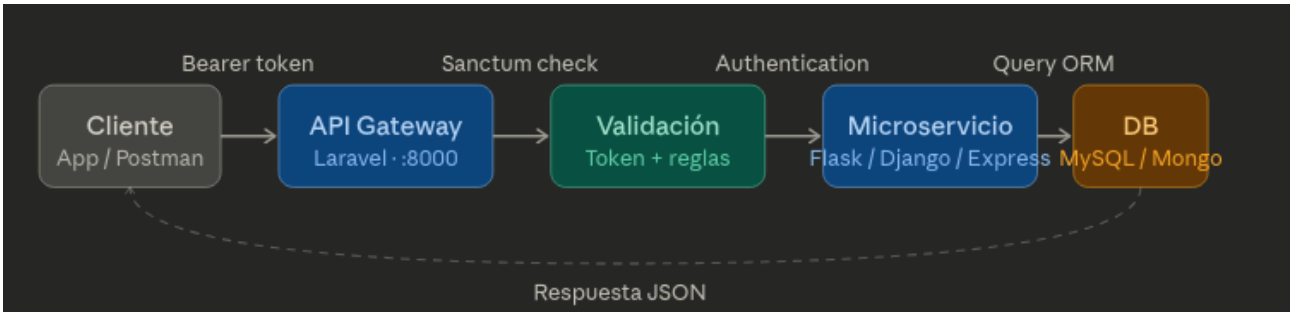
1.1 Microservicios del Sistema

Microservicio	Framework	Base de Datos	Puerto	Estado
Gateway + Usuarios	Laravel 11	MySQL	:8000	Activo
Cursos	Flask	MySQL	:5000	Activo
Contenido	Express (Node.js)	MongoDB	:3000	Activo
Inscripciones	Django	PostgreSQL	:8080	Activo
Evaluaciones	Flask	MySQL	:5001	Activo

1.2 Diagrama de Arquitectura



1.3 Flujo de una Petición



Flujo: Cliente → Gateway → Validación → Microservicio → Base de Datos → respuesta JSON

El Gateway es el único punto de entrada. Valida el token Sanctum del cliente, aplica reglas de negocio (verificar que el curso exista antes de inscribir, etc.) y luego delega la petición al microservicio correspondiente usando el header interno

2. Autenticación y Seguridad

2.1 Autenticación de Clientes — Laravel Sanctum

El Gateway usa Laravel Sanctum para manejar la autenticación de usuarios externos. Al hacer login el cliente recibe un token Bearer que debe incluir en todas las peticiones posteriores.

Flujo de autenticación:

1. El cliente envía credenciales a POST /api/login
2. El Gateway valida con Sanctum y retorna un token Bearer
3. El cliente incluye el token en el header de cada petición
4. El Gateway valida el token antes de delegar al microservicio

Header requerido en todas las peticiones al Gateway:

```
Authorization: Bearer <token>
```

2.2 Token Interno entre Microservicios

La comunicación interna entre el Gateway y los microservicios usa un header propio llamado Authentication. Contiene una clave secreta compartida (SERVICES_TOKEN) que cada microservicio valida antes de procesar la petición.

Header interno:

```
Authentication: <SERVICES_TOKEN>
```

Los microservicios deben rechazar cualquier petición que no incluya el header Authentication con la clave correcta. Esto garantiza que solo el Gateway pueda comunicarse con ellos.

2.3 Validaciones del Gateway

Acción	Validación
Crear inscripción	Verifica que el curso exista antes de inscribir al usuario
Crear módulo	Verifica que el curso exista antes de crear el módulo
Crear lección	Verifica que el módulo exista antes de crear la lección

Crear quiz	Verifica que el curso exista antes de crear el quiz
Enviar respuestas	Verifica que el quiz exista y que el usuario esté inscrito
Crear curso	Verifica que el instructor_id corresponda a un usuario válido

3. API Gateway + Usuarios (Laravel 11 — Puerto 8000)

3.1 Instalación de Dependencias

```
cd api_gateway
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate
php artisan serve --port=8000
```

3.2 Variables de Entorno (.env)

Campo	Tipo	Descripción / Valor
APP_NAME	string	AcademiX
APP_URL	string	http://localhost:8000
DB_CONNECTION	string	mysql
DB_HOST	string	127.0.0.1
DB_PORT	int	3306
DB_DATABASE	string	academix_gateway
DB_USERNAME	string	root
DB_PASSWORD	string	tu_contraseña
SERVICES_TOKEN	string	clave_secreta_interna (misma en todos los servicios)
COURSES_SERVICE_URL	string	http://localhost:5000
CONTENT_SERVICE_URL	string	http://localhost:3000
ENROLLMENTS_SERVICE_URL	string	http://localhost:8080
EVALUATIONS_SERVI	string	http://localhost:5001

CE_URL		

3.3 Tabla: users

Campo	Tipo	Descripción
id	INT PK	Identificador único autoincremental
name	VARCHAR(100)	Nombre completo del usuario
email	VARCHAR(150)	Correo electrónico (único)
password	VARCHAR(255)	Hash de contraseña (bcrypt)
question	VARCHAR(100)	Pregunta de seguridad para recuperación
answer	VARCHAR(100)	Respuesta a la pregunta de seguridad

3.4 Endpoints

Autenticación

Método	Ruta	Descripción
POST	`/api/login`	Iniciar sesión y obtener token Bearer
POST	`/api/logout`	Cerrar sesión e invalidar token
POST	`/api/restore_password`	Recuperación de contraseña vía pregunta de seguridad

Cursos (proxy → courses_service)

Método	Ruta	Descripción
GET	`/api/courses`	Listar todos los cursos publicados
GET	`/api/courses/{id}`	Obtener detalle de un curso
POST	`/api/courses`	Crear un nuevo curso (requiere id del usuario)
PUT	`/api/courses/{id}`	Editar un curso existente
DELETE	`/api/courses/{id}`	Eliminar un curso

Módulos (proxy → content_service)

Método	Ruta	Descripción
GET	`/api/modules`	Listar módulos de los cursos
POST	`/api/modules`	Crear un módulo (valida que el curso exista)
PUT	`/api/modules/{id}`	Editar un módulo
DELETE	`/api/modules/{id}`	Eliminar un módulo

Lecciones (proxy → content_service)

Método	Ruta	Descripción
GET	`/api/lessons`	Obtener todas las lecciones
GET	`/api/lessons/{id}`	Obtener detalle de una lección
POST	`/api/lessons`	Crear una lección (valida que el módulo exista)
PUT	`/api/lessons/{id}`	Editar una lección
DELETE	`/api/lessons/{id}`	Eliminar una lección

Inscripciones (proxy → enrollments_service)

Método	Ruta	Descripción
POST	`/api/enrollments`	Inscribirse a un curso (valida que el curso exista)
GET	`/api/enrollments`	Obtener todas las inscripciones
PUT	`/api/enrollments/{id}`	Marcar inscripción como completada / editar
DELETE	`/api/enrollments/{id}`	Cancelar inscripción

Evaluaciones (proxy → evaluations_service)

Método	Ruta	Descripción
GET	`/api/exams`	Listar todos los quizzes
POST	`/api/exams`	Crear un quiz (valida que el curso exista)
PUT	`/api/exams/{id}`	Editar un quiz
DELETE	`/api/exams/{id}`	Eliminar un quiz
GET	`/api/questions`	Listar preguntas
POST	`/api/questions`	Crear pregunta (valida que el quiz exista)

PUT	`/api/questions/{id}`	Editar una pregunta
DELETE	`/api/questions/{id}`	Eliminar una pregunta
POST	`/api/submissions`	Enviar respuestas (valida inscripción del usuario)
GET	`/api/submissions`	Ver envíos de respuestas
PUT	`/api/submissions/{id}`	Editar respuestas
DELETE	`/api/submissions/{id}`	Eliminar una respuesta

4. Courses Service (Flask — Puerto 5000)

4.1 Instalación de Dependencias

```
cd courses_service
python -m venv venv
source venv/bin/activate        # Linux/Mac
venv\Scripts\activate           # Windows
pip install -r requirements.txt
flask db upgrade
flask run --port=5000
```

4.2 Variables de Entorno (.env)

Campo	Tipo	Descripción / Valor
FLASK_APP	string	app.py
FLASK_ENV	string	development
DATABASE_URL	string	mysql+pymysql://root:@localhost/academix_courses
SERVICES_TOKEN	string	clave_secreta_interna (debe coincidir con el Gateway)

4.3 Tabla: courses

Campo	Tipo	Descripción
id	INT PK	Identificador único autoincremental
title	VARCHAR(200)	Título del curso
description	TEXT	Descripción detallada del curso

category	VARCHAR(100)	Categoría del curso (ej: Programación, Diseño)
level	VARCHAR(20)	beginner | intermediate | advanced
instructor_id	INT	ID del instructor (enviado por el Gateway)

4.4 Endpoints

Método	Ruta	Descripción
GET	`/api/courses`	Listar todos los cursos
GET	`/api/courses/{id}`	Obtener detalle de un curso específico
POST	`/api/courses`	Crear un nuevo curso
PUT	`/api/courses/{id}`	Editar un curso existente
DELETE	`/api/courses/{id}`	Eliminar un curso

5. Content Service (Express/Node.js — Puerto 3000)

5.1 Instalación de Dependencias

```
cd content_service
npm install
node server.js
```

5.2 Variables de Entorno (.env)

Campo	Tipo	Descripción / Valor
PORT	int	3000
MONGO_URI	string	mongodb://localhost:27017/academix_content
SERVICES_TOKEN	string	clave_secreta_interna (debe coincidir con el Gateway)

5.3 Colecciones MongoDB

modules

Campo	Tipo	Descripción
_id	ObjectId	Identificador único generado por MongoDB
course_id	INT	ID del curso al que pertenece el módulo
title	String	Título del módulo
description	String	Descripción del módulo

lessons

Campo	Tipo	Descripción
_id	ObjectId	Identificador único generado por MongoDB
module_id	ObjectId	ID del módulo al que pertenece la lección
title	String	Título de la lección
type	String	video | text | file | link
content	Object	Objeto flexible según el tipo de lección

5.4 Endpoints

Módulos

Método	Ruta	Descripción
GET	`/api/modules`	Listar módulos de un curso
POST	`/api/modules`	Crear un nuevo módulo
PUT	`/api/modules/:id`	Editar un módulo existente
DELETE	`/api/modules/:id`	Eliminar un módulo

Lecciones

Método	Ruta	Descripción
GET	`/api/lessons/:id`	Obtener detalle completo de una lección
POST	`/api/lessons`	Crear una lección (cualquier tipo)
PUT	`/api/lessons/:id`	Editar una lección
DELETE	`/api/lessons/:id`	Eliminar una lección

6. Enrollments Service (Django — Puerto 8080)

6.1 Instalación de Dependencias

```
cd enrollments_service
python -m venv venv
source venv/bin/activate        # Linux/Mac
venv\Scripts\activate           # Windows
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver 8080
```

6.2 Variables de Entorno (.env)

Campo	Tipo	Descripción / Valor
DB_NAME	string	academix_enrollments
DB_USER	string	tu_usuario_postgres
DB_PASSWORD	string	tu_contraseña
DB_HOST	string	localhost
DB_PORT	int	5432
SERVICES_TOKEN	string	clave_secreta_interna (debe coincidir con el Gateway)

6.3 Tabla: enrollments

Campo	Tipo	Descripción
id	SERIAL PK	Identificador único autoincremental
user_id	INT	ID del usuario (enviado por el Gateway)
course_id	INT	ID del curso (referencia al courses_service)
status	VARCHAR(20)	active | completed | cancelled

6.4 Endpoints

Método	Ruta	Descripción
POST	`/api/enrollments/`	Inscribirse a un curso

GET	`/api/enrollments/`	Listar todas las inscripciones
GET	`/api/enrollments/{id}`	Obtener una inscripción específica
PUT	`/api/enrollments/{id}`	Editar inscripción (cambiar status)
DELETE	`/api/enrollments/{id}/`	Cancelar inscripción

7. Evaluations Service (Flask — Puerto 5001)

7.1 Instalación de Dependencias

```
cd evaluations_service
python -m venv venv
source venv/bin/activate        # Linux/Mac
venv\Scripts\activate           # Windows
pip install -r requirements.txt
flask db upgrade
flask run --port=5001
```

7.2 Variables de Entorno (.env)

Campo	Tipo	Descripción / Valor
FLASK_APP	string	app.py
FLASK_ENV	string	development
DATABASE_URL	string	mysql+pymysql://user:password@localhost:3306/academix_evaluations
SERVICES_TOKEN	string	clave_secreta_interna (debe coincidir con el Gateway)

7.3 Tablas

exams

Campo	Tipo	Descripción
id	INT PK	Identificador único autoincremental
course_id	INT	ID del curso (el Gateway valida que exista)
title	VARCHAR(200)	Título del quiz/examen
passing_score	INT	Puntaje mínimo para aprobar (ej: 10)

questions

Campo	Tipo	Descripción
id	INT PK	Identificador único autoincremental
quiz_id	INT FK	Quiz al que pertenece la pregunta
text	TEXT	Enunciado de la pregunta
option_a	VARCHAR(255)	Opción A
option_b	VARCHAR(255)	Opción B
option_c	VARCHAR(255)	Opción C
option_d	VARCHAR(255)	Opción D
correct_option	CHAR(1)	Opción correcta: a, b, c o d

submissions

Campo	Tipo	Descripción
id	INT PK	Identificador único autoincremental
user_id	INT	ID del usuario (enviado por el Gateway)
exam_id	INT FK	Examen respondido (el Gateway valida que exista)
score	INT	Puntaje obtenido en la evaluación
passed	BOOLEAN	True si el score supera el passing_score

7.4 Endpoints

Exámenes

Método	Ruta	Descripción
GET	`/api/exams`	Listar quizzes (incluye sus preguntas)
POST	`/api/exams`	Crear un nuevo quiz
PUT	`/api/exams/{id}`	Editar un quiz
DELETE	`/api/exams/{id}`	Eliminar un quiz

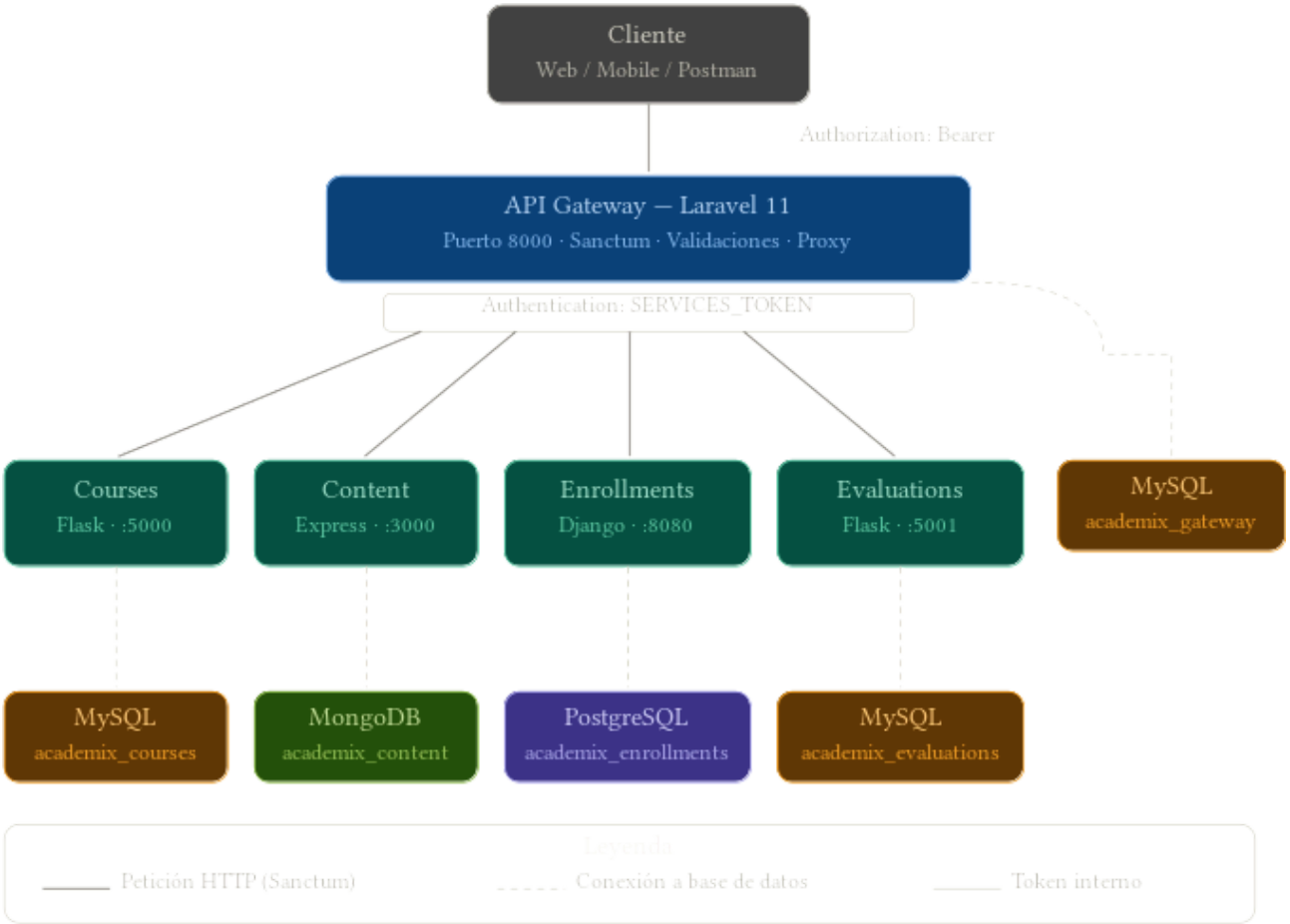
Preguntas

Método	Ruta	Descripción
GET	`/api/questions`	Listar preguntas
POST	`/api/questions`	Crear una pregunta de opción múltiple
PUT	`/api/questions/{id}`	Editar una pregunta
DELETE	`/api/questions/{id}`	Eliminar una pregunta

Submissions

Método	Ruta	Descripción
POST	`/api/submissions`	Enviar respuestas y recibir calificación automática
GET	`/api/submissions`	Ver todos los envíos de respuestas
PUT	`/api/submissions/{id}`	Edita un envio de respuestas
DELETE	`/api/submissions/{id}`	Elimina un envio de respuestas

8. Diagrama de arquitectura



Relaciones entre servicios: Los microservicios no tienen Foreign Keys entre bases de datos. El API Gateway es responsable de validar la integridad referencial antes de delegar cada petición.

9. Estructura del Repositorio

```
academix/
├── api_gateway/              # API Gateway (Laravel 11)
│   ├── app/Http/Controllers/ # CourseController, EnrollmentController, etc.
│   ├── routes/api.php        # Definición de rutas
│   └── .env.example
├── courses_service/          # Microservicio de Cursos (Flask)
│   ├── app.py
│   ├── requirements.txt
│   └── .env.example
├── content_service/          # Microservicio de Contenido (Express)
│   ├── server.js
│   ├── package.json
│   └── .env.example
├── enrollments_service/      # Microservicio de Inscripciones (Django)
│   ├── manage.py
│   ├── requirements.txt
│   └── .env.example
├── evaluations_service/      # Microservicio de Evaluaciones (Flask)
│   ├── app.py
│   ├── requirements.txt
│   └── .env.example
├── tests/                   # Almacenamiento de pruebas
│   ├──perfomance/
├── docs/                    # Carpeta para guardar los documentos
│   ├──documentacion.pdf/
└── README.md
```

10. Pruebas de rendimiento

Las pruebas de rendimiento del sistema AcademiX se realizaron utilizando la herramienta **Locust**, con el objetivo de evaluar el comportamiento de la aplicación bajo diferentes niveles de carga.

Las pruebas se ejecutaron a través del **API Gateway**, simulando el flujo real de uso del sistema. Cada usuario virtual inicia sesión mediante el proceso de autenticación (login) y, a partir de allí, realiza distintas operaciones como consultas, creación, actualización y eliminación de recursos en los microservicios.

Se definieron distintos escenarios variando la cantidad de usuarios concurrentes y la tasa de generación, lo que permitió analizar el desempeño del sistema en condiciones normales y bajo alta demanda.

Para ejecutar las pruebas, es necesario tener **Python instalado** y posteriormente instalar Locust con el siguiente comando:

pip install locust

Una vez instalado, las pruebas pueden ejecutarse desde la carpeta correspondiente utilizando un archivo `locustfile`, por ejemplo:

locust -f tests/performance/courses/locustfile_courses.py --host=http://localhost:8000

Luego, se debe acceder a la interfaz web de Locust en el navegador:

http://localhost:8089

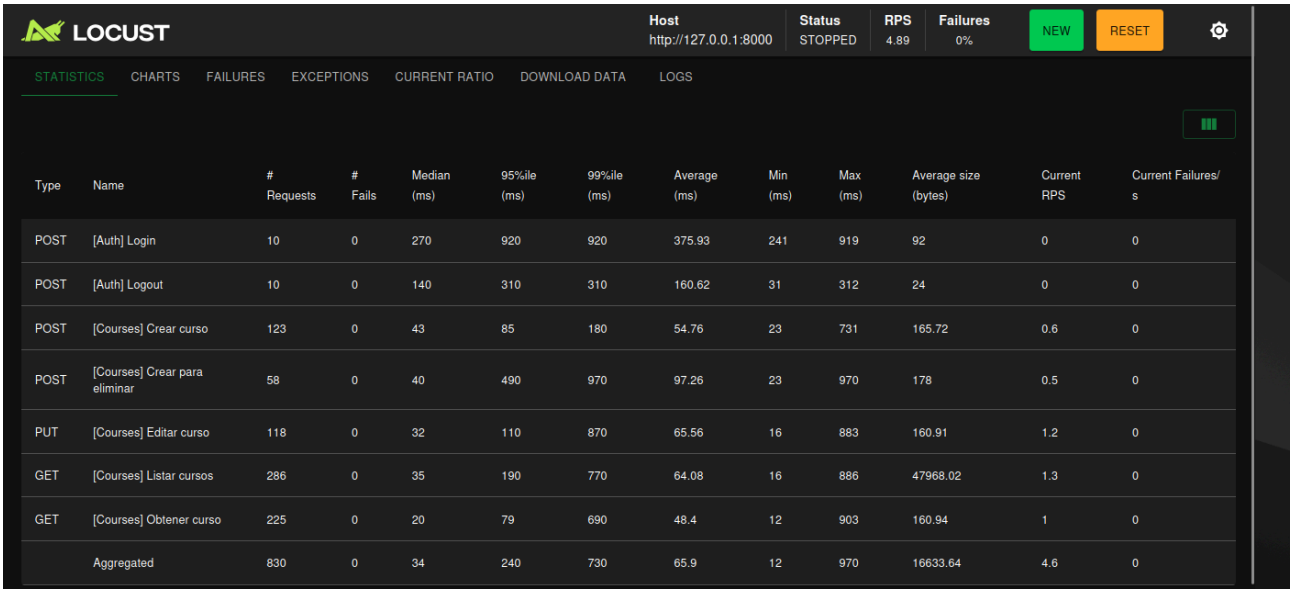
Desde allí se configuran los parámetros de la prueba, como el número de usuarios concurrentes y la tasa de generación de los mismos.

Los resultados obtenidos se presentan a continuación junto con su respectivo análisis para cada microservicio.

10.1 Resultados – Microservicio Courses

A continuación, se presentan los resultados obtenidos para el microservicio de **Courses**, evaluando su comportamiento en tres escenarios: capacidad, carga y estrés.

- **Prueba de Capacidad**



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	10	0	270	920	920	375.93	241	919	92	0	0
POST	[Auth] Logout	10	0	140	310	310	160.62	31	312	24	0	0
POST	[Courses] Crear curso	123	0	43	85	180	54.76	23	731	165.72	0.6	0
POST	[Courses] Crear para eliminar	58	0	40	490	970	97.26	23	970	178	0.5	0
PUT	[Courses] Editar curso	118	0	32	110	870	65.56	16	883	160.91	1.2	0
GET	[Courses] Listar cursos	286	0	35	190	770	64.08	16	886	47968.02	1.3	0
GET	[Courses] Obtener curso	225	0	20	79	690	48.4	12	903	160.94	1	0
	Aggregated	830	0	34	240	730	65.9	12	970	16633.64	4.6	0

En este escenario se evaluó el sistema con una carga baja (10 usuarios concurrentes), con el objetivo de analizar su comportamiento en condiciones normales.

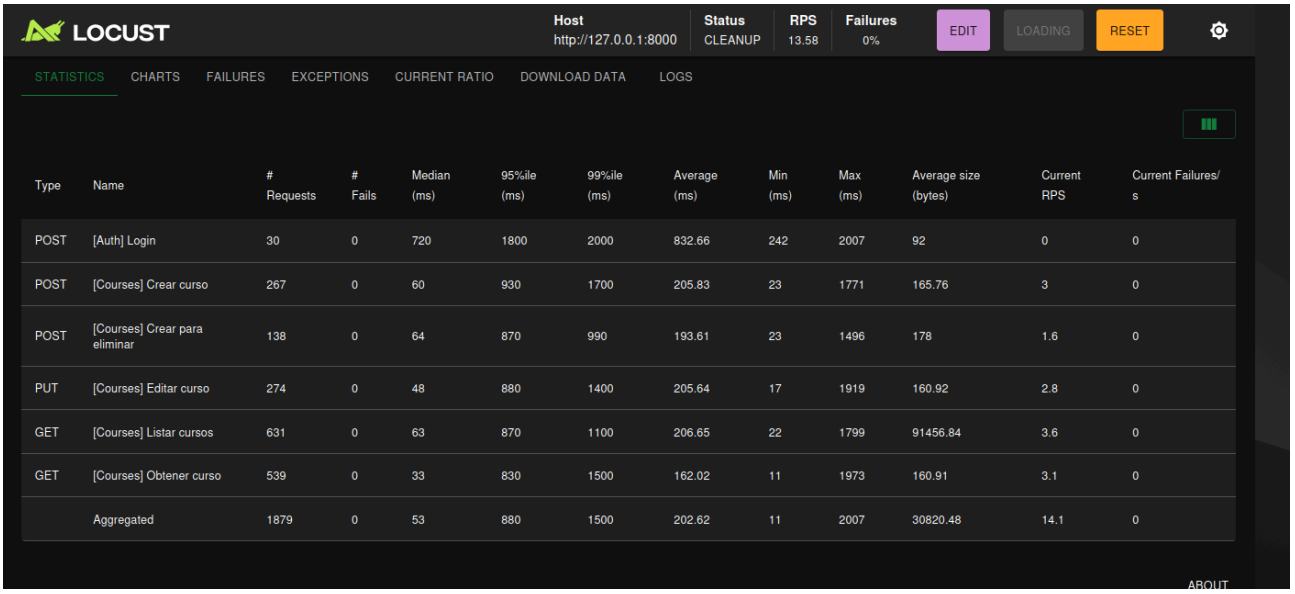
Los resultados muestran un **RPS estable cercano a 4.8 solicitudes por segundo**, sin presencia de errores (0% de fallos), lo cual indica un correcto funcionamiento del sistema en condiciones básicas.

En cuanto a los tiempos de respuesta, se observa que:

- El tiempo promedio se mantiene alrededor de **65 ms**
- El percentil 95 alcanza valores cercanos a **240 ms**
- Existen picos aislados de hasta **900 ms**, aunque no son constantes

Esto indica que el sistema responde de manera rápida y estable en condiciones normales, con ligeras variaciones que no afectan significativamente la experiencia del usuario.

- **Prueba de Carga**



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	30	0	720	1800	2000	832.66	242	2007	92	0	0
POST	[Courses] Crear curso	267	0	60	930	1700	205.83	23	1771	165.76	3	0
POST	[Courses] Crear para eliminar	138	0	64	870	990	193.61	23	1496	178	1.6	0
PUT	[Courses] Editar curso	274	0	48	880	1400	205.64	17	1919	160.92	2.8	0
GET	[Courses] Listar cursos	631	0	63	870	1100	206.65	22	1799	91456.84	3.6	0
GET	[Courses] Obtener curso	539	0	33	830	1500	162.02	11	1973	160.91	3.1	0
	Aggregated	1879	0	53	880	1500	202.62	11	2007	30820.48	14.1	0

Para este escenario se incrementó el número de usuarios concurrentes a 30, con el fin de evaluar el comportamiento del sistema bajo una demanda moderada.

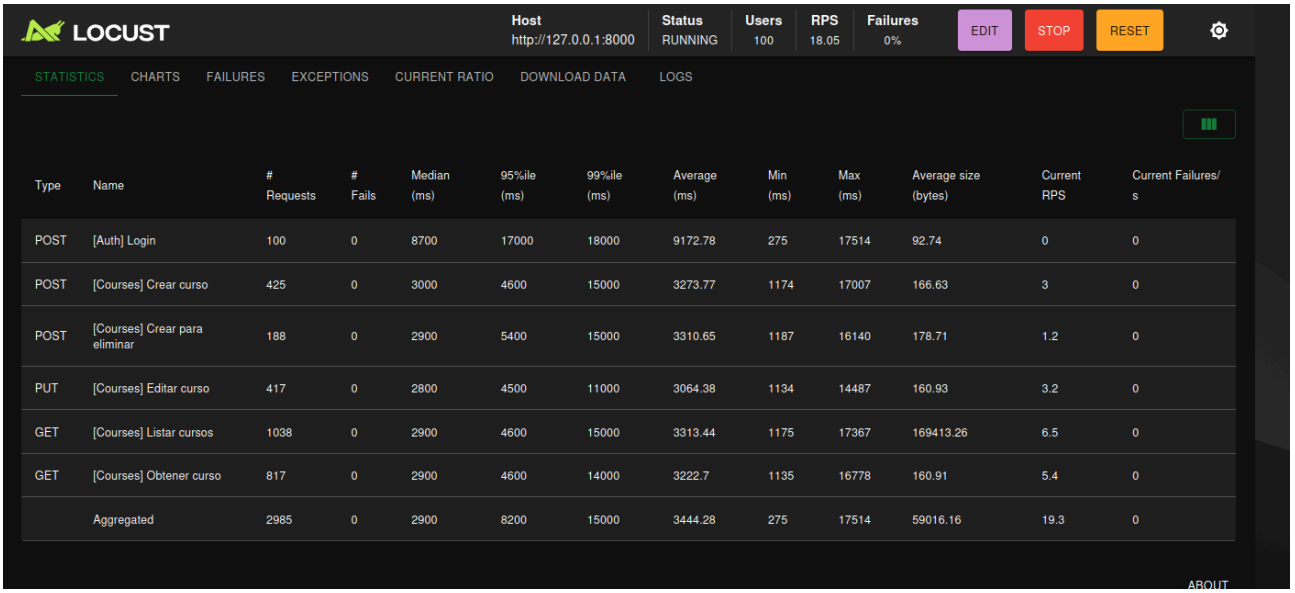
El sistema logró mantener un **RPS aproximado de 14 solicitudes por segundo**, nuevamente sin errores registrados, lo cual evidencia una buena capacidad de manejo de múltiples peticiones simultáneas.

Sin embargo, los tiempos de respuesta presentan un incremento notable:

- Promedio cercano a **200 ms**
- Percentil 95 alrededor de **880 ms**
- Picos de hasta **2000 ms**

Estos resultados indican que, aunque el sistema sigue siendo estable, comienza a presentar **mayor latencia bajo carga**, lo cual es esperado en este tipo de escenarios.

- **Prueba de Estrés**



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	100	0	8700	17000	18000	9172.78	275	17514	92.74	0	0
POST	[Courses] Crear curso	425	0	3000	4600	15000	3273.77	1174	17007	166.63	3	0
POST	[Courses] Crear para eliminar	188	0	2900	5400	15000	3310.65	1187	16140	178.71	1.2	0
PUT	[Courses] Editar curso	417	0	2800	4500	11000	3064.38	1134	14487	160.93	3.2	0
GET	[Courses] Listar cursos	1038	0	2900	4600	15000	3313.44	1175	17367	169413.26	6.5	0
GET	[Courses] Obtener curso	817	0	2900	4600	14000	3222.7	1135	16778	160.91	5.4	0
	Aggregated	2985	0	2900	8200	15000	3444.28	275	17514	59016.16	19.3	0

En la prueba de estrés se simularon hasta 100 usuarios concurrentes, con el objetivo de llevar el sistema a sus límites.

Se observa que el sistema alcanza aproximadamente **17–18 RPS**, manteniendo un **0% de fallos**, lo cual es un indicador positivo en términos de estabilidad.

No obstante, los tiempos de respuesta aumentan considerablemente:

- Picos iniciales superiores a **15,000 ms**
- Posterior estabilización entre **3,000 ms y 4,000 ms**

Esto evidencia que el sistema, aunque no falla, **pierde capacidad de respuesta bajo alta carga**, generando tiempos de espera elevados que impactarían negativamente la experiencia del usuario en un entorno real.

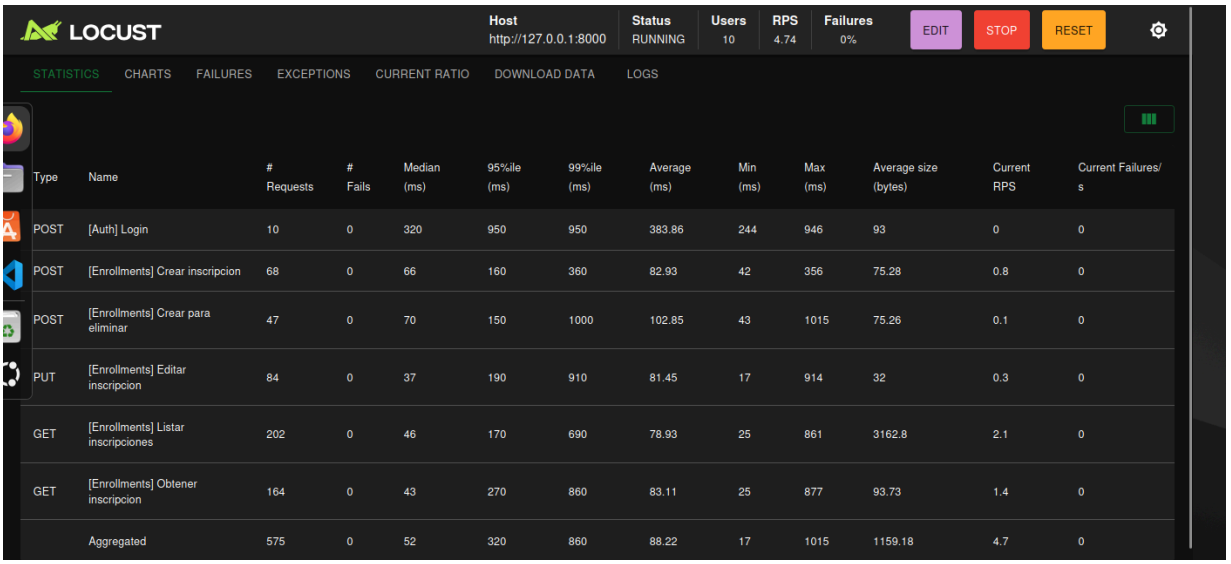
El microservicio de Courses presenta un comportamiento:

- **Estable y eficiente** en condiciones normales (capacidad)
- **Aceptable pero con aumento de latencia** en carga moderada
- **Degradado en tiempos de respuesta** bajo condiciones de estrés, aunque sin fallos

En general, el sistema demuestra una buena tolerancia a la carga, pero requiere optimizaciones a nivel de rendimiento (consultas, concurrencia o escalabilidad) para mejorar los tiempos de respuesta en escenarios de alta demanda.

10.2 Resultados – Microservicio Enrollments

- **Prueba de capacidad:** En este escenario se evaluó el microservicio de Enrollments con 10 usuarios concurrentes, verificando la estabilidad del servicio de inscripciones en condiciones normales de uso.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	10	0	320	950	950	383.86	244	946	93	0	0
POST	[Enrollments] Crear inscripcion	68	0	66	160	360	82.93	42	356	75.28	0.8	0
POST	[Enrollments] Crear para eliminar	47	0	70	150	1000	102.85	43	1015	75.26	0.1	0
PUT	[Enrollments] Editar inscripcion	84	0	37	190	910	81.45	17	914	32	0.3	0
GET	[Enrollments] Listar inscripciones	202	0	46	170	690	78.93	25	861	3162.8	2.1	0
GET	[Enrollments] Obtener inscripcion	164	0	43	270	860	83.11	25	877	93.73	1.4	0
	Aggregated	575	0	52	320	860	88.22	17	1015	1159.18	4.7	0

Los resultados muestran un RPS estable de 4.74 solicitudes por segundo, sin presencia de errores (0% de fallos). La gráfica de RPS muestra una curva ascendente suave que se estabiliza rápidamente, y los tiempos de respuesta se mantienen dentro de rangos aceptables durante toda la prueba.

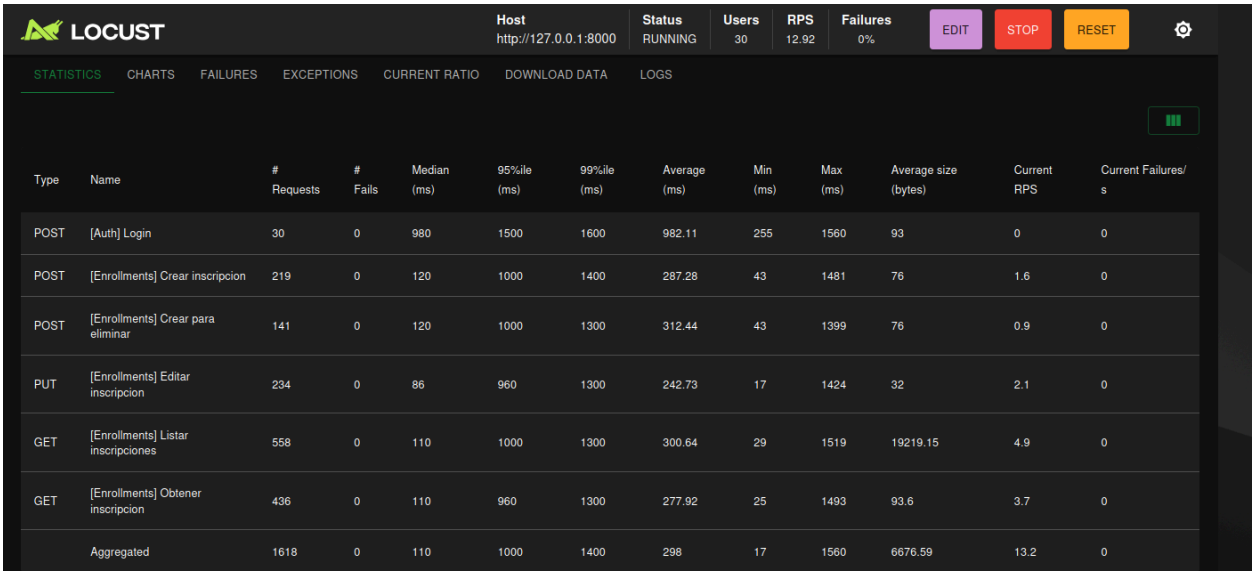
En cuanto a los tiempos de respuesta, se observa que:

- El tiempo promedio general se mantiene en 88 ms
- El percentil 95 alcanza valores cercanos a 320 ms

- Las operaciones de lectura (listar, obtener) presentan medianas de 46 ms y 43 ms respectivamente
- Existen picos aislados de hasta 1015 ms en la operacion de eliminacion, aunque no son constantes

Esto indica que el sistema responde de manera rapida y estable bajo carga baja, confirmando el correcto funcionamiento del microservicio en condiciones normales.

- **Prueba de carga:** Para este escenario se incremento el numero de usuarios concurrentes a 30, con el fin de evaluar el comportamiento del microservicio de Enrollments bajo una demanda moderada.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	30	0	980	1500	1600	982.11	255	1560	93	0	0
POST	[Enrollments] Crear inscripcion	219	0	120	1000	1400	287.28	43	1481	76	1.6	0
POST	[Enrollments] Crear para eliminar	141	0	120	1000	1300	312.44	43	1399	76	0.9	0
PUT	[Enrollments] Editar inscripcion	234	0	86	960	1300	242.73	17	1424	32	2.1	0
GET	[Enrollments] Listar inscripciones	558	0	110	1000	1300	300.64	29	1519	19219.15	4.9	0
GET	[Enrollments] Obtener inscripcion	436	0	110	960	1300	277.92	25	1493	93.6	3.7	0
	Aggregated	1618	0	110	1000	1400	298	17	1560	6676.59	13.2	0

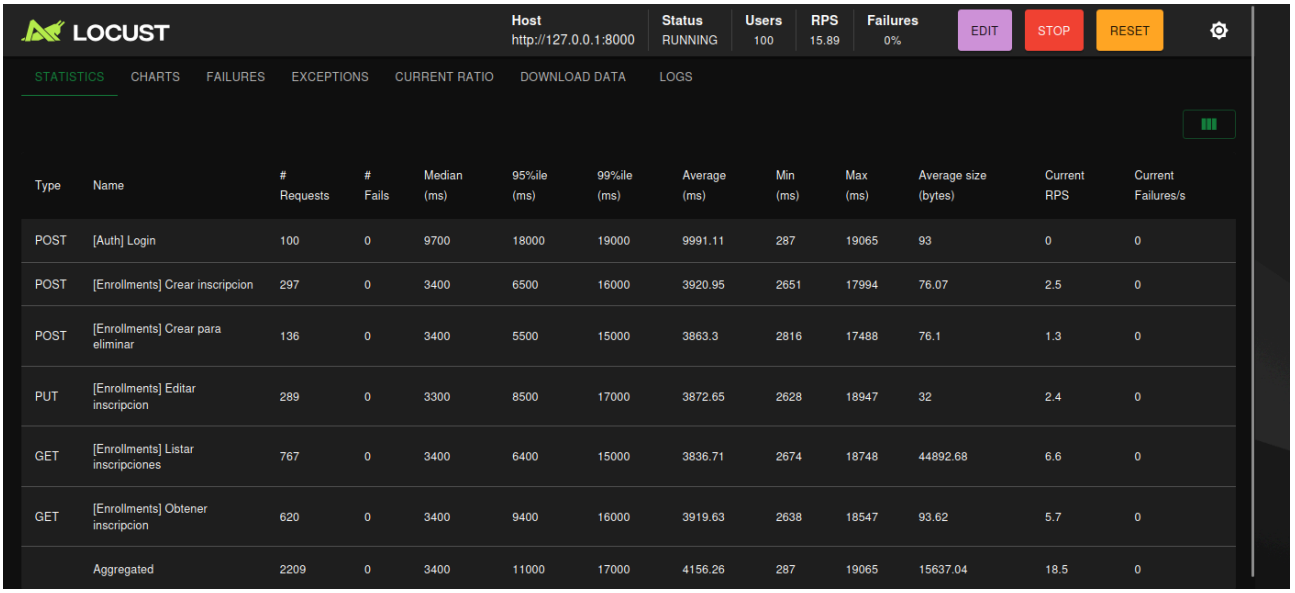
El sistema logro mantener un RPS aproximado de 12.92 solicitudes por segundo, nuevamente sin errores registrados, lo cual evidencia una buena capacidad de manejo de multiples peticiones simultaneas. La gráfica muestra un pico inicial de tiempos (~1500 ms en p95) durante el spawn de usuarios que luego se estabiliza.

Sin embargo, los tiempos de respuesta presentan un incremento notable respecto a la prueba de capacidad:

- Promedio general cercano a 298 ms, frente a los 88 ms de capacidad
- Percentil 95 alrededor de 1000 ms
- El login presenta un p95 de 1500 ms, mostrando mayor impacto bajo carga moderada
- Picos de hasta 1560 ms en algunas operaciones de escritura

Estos resultados indican que el microservicio sigue siendo estable bajo carga moderada, aunque comienza a presentar mayor latencia, comportamiento esperado en este tipo de escenarios.

- **Prueba de Estres:** En la prueba de estres se simularon hasta 100 usuarios concurrentes, llevando el microservicio de Enrollments a sus limites operacionales.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	100	0	9700	18000	19000	9991.11	287	19065	93	0	0
POST	[Enrollments] Crear inscripcion	297	0	3400	6500	16000	3920.95	2651	17994	76.07	2.5	0
POST	[Enrollments] Crear para eliminar	136	0	3400	5500	15000	3863.3	2816	17488	76.1	1.3	0
PUT	[Enrollments] Editar inscripcion	289	0	3300	8500	17000	3872.65	2628	18947	32	2.4	0
GET	[Enrollments] Listar inscripciones	767	0	3400	6400	15000	3836.71	2674	18748	44892.68	6.6	0
GET	[Enrollments] Obtener inscripcion	620	0	3400	9400	16000	3919.63	2638	18547	93.62	5.7	0
	Aggregated	2209	0	3400	11000	17000	4156.26	287	19065	15637.04	18.5	0

Se observa que el sistema alcanza aproximadamente 15.89 RPS, manteniendo un 0% de fallos, lo cual es un indicador positivo de estabilidad. Sin embargo, la gráfica muestra el patron caracteristico de estres: un pico inicial superior a 18,000 ms al momento del spawn masivo, seguido de estabilizacion alrededor de los 3,000-4,000 ms.

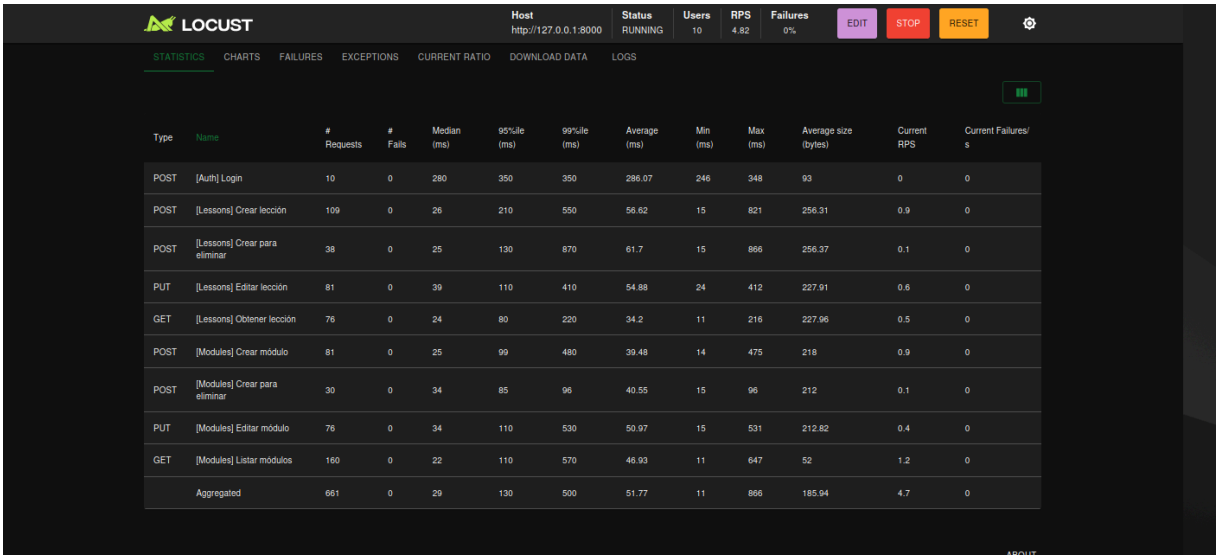
Los tiempos de respuesta bajo estres muestran una degradacion significativa:

- Mediana general de 3,400 ms frente a los 52 ms de la prueba de capacidad
- Percentil 95 de 11,000 ms con picos de hasta 19,065 ms
- El login se convierte en el cuello de botella con un promedio de 9,991 ms y p95 de 18,000 ms
- Las operaciones de inscripcion y edicion superan los 3,000 ms de mediana

Esto evidencia que el microservicio, aunque no falla bajo alta carga, pierde capacidad de respuesta de forma significativa, generando tiempos de espera que impactarian negativamente la experiencia del usuario en un entorno real.

10.3 Resultados – Microservicio Content

- **Prueba de capacidad:** En este escenario se evaluo el microservicio de Content con 10 usuarios concurrentes, cubriendo operaciones sobre modulos y lecciones en condiciones normales de uso.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	10	0	280	350	350	286.07	246	348	93	0	0
POST	[Lessons] Crear lección	109	0	26	210	550	56.62	15	821	256.31	0.9	0
POST	[Lessons] Crear para eliminar	38	0	25	130	870	61.7	15	866	256.37	0.1	0
PUT	[Lessons] Editar lección	81	0	39	110	410	54.88	24	412	227.91	0.6	0
GET	[Lessons] Obtener lección	76	0	24	80	220	34.2	11	216	227.96	0.5	0
POST	[Modules] Crear módulo	81	0	25	99	480	39.48	14	475	218	0.9	0
POST	[Modules] Crear para eliminar	30	0	34	85	96	40.55	15	96	212	0.1	0
PUT	[Modules] Editar módulo	76	0	34	110	530	50.97	15	531	212.82	0.4	0
GET	[Modules] Listar módulos	160	0	22	110	570	46.93	11	647	52	1.2	0
	Aggregated	661	0	29	130	500	51.77	11	866	185.94	4.7	0

Los resultados muestran un RPS estable de 4.82 solicitudes por segundo, sin presencia de errores (0% de fallos). Este microservicio presento los tiempos de respuesta mas bajos de todo el

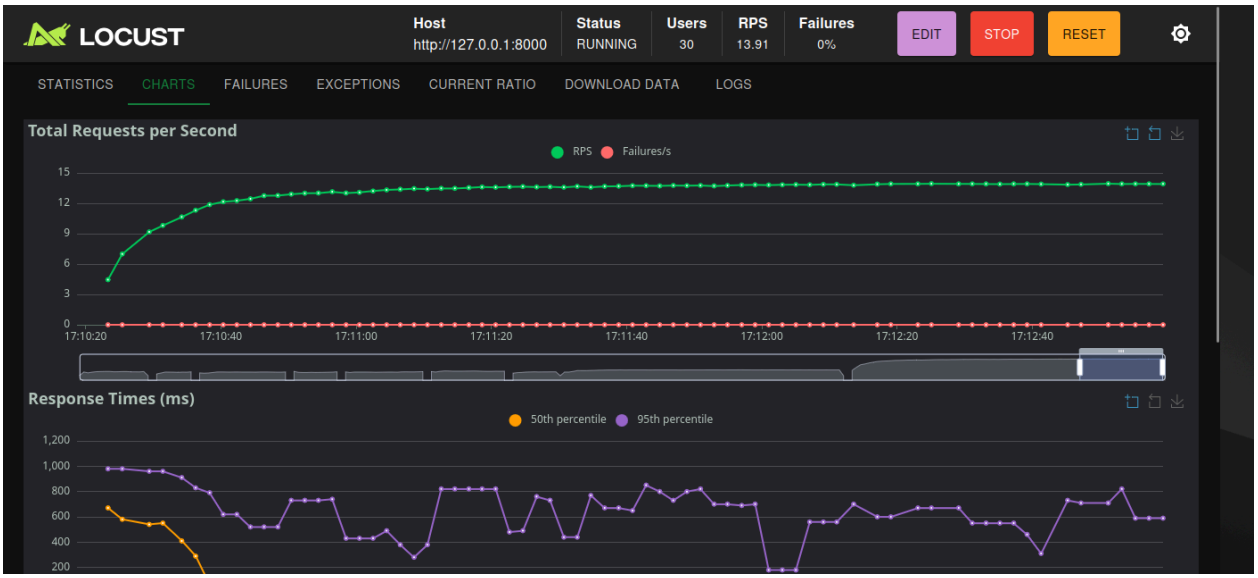
sistema, gracias a la naturaleza flexible de MongoDB para consultas de contenido variado.

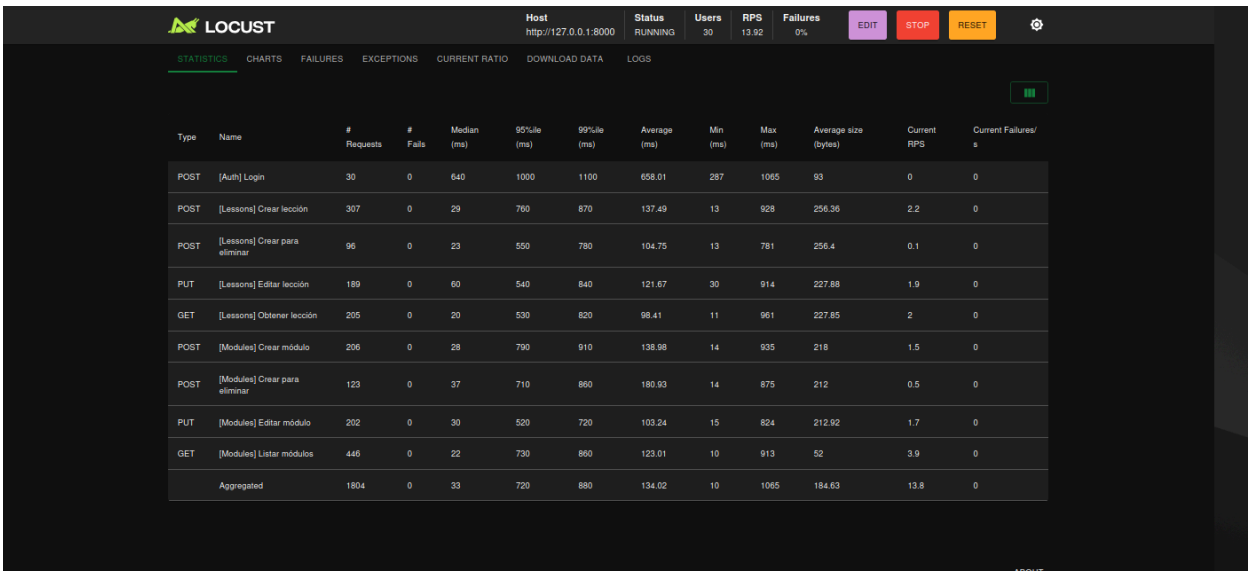
En cuanto a los tiempos de respuesta, se observa que:

- El tiempo promedio general se mantiene en 51.77 ms, el mas bajo del sistema
- El percentil 95 alcanza solo 130 ms
- Las operaciones de lectura presentan medianas de 22 ms (listar modulos) y 24 ms (obtener leccion)
- Todas las operaciones de escritura se mantienen por debajo de 40 ms en mediana

Esto confirma que MongoDB es una eleccion adecuada para el contenido flexible del sistema, ofreciendo tiempos de respuesta muy competitivos bajo carga baja.

- **Prueba de carga:** Para este escenario se incremento el numero de usuarios concurrentes a 30, evaluando el comportamiento del Content Service bajo demanda moderada.





Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	30	0	640	1000	1100	658.01	287	1065	93	0	0
POST	[Lessons] Crear lección	307	0	29	760	870	137.49	13	928	256.36	2.2	0
POST	[Lessons] Crear para eliminar	96	0	23	550	780	104.75	13	781	256.4	0.1	0
PUT	[Lessons] Editar lección	189	0	60	540	840	121.67	30	914	227.88	1.9	0
GET	[Lessons] Obtener lección	205	0	20	530	820	98.41	11	961	227.85	2	0
POST	[Modules] Crear módulo	206	0	28	790	910	138.98	14	935	218	1.5	0
POST	[Modules] Crear para eliminar	123	0	37	710	860	180.93	14	875	212	0.5	0
PUT	[Modules] Editar módulo	202	0	30	520	720	103.24	15	824	212.92	1.7	0
GET	[Modules] Listar módulos	446	0	22	730	860	123.01	10	913	52	3.9	0
	Aggregated	1804	0	33	720	880	134.02	10	1065	184.63	13.8	0

El sistema logro mantener un RPS de 13.92 solicitudes por segundo sin registrar errores. La gráfica muestra una curva de RPS muy limpia y estable que se estabiliza rapidamente, con tiempos que descienden progresivamente desde un pico inicial (~1000 ms) hasta mantenerse

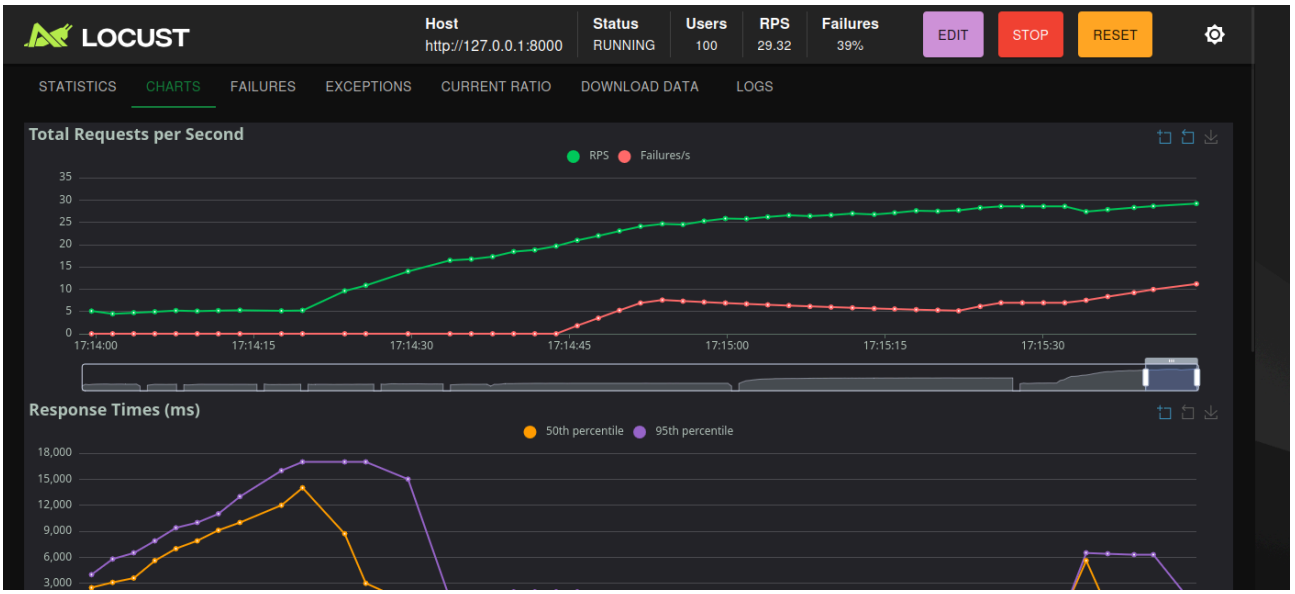
alrededor de 500-700 ms en p95.

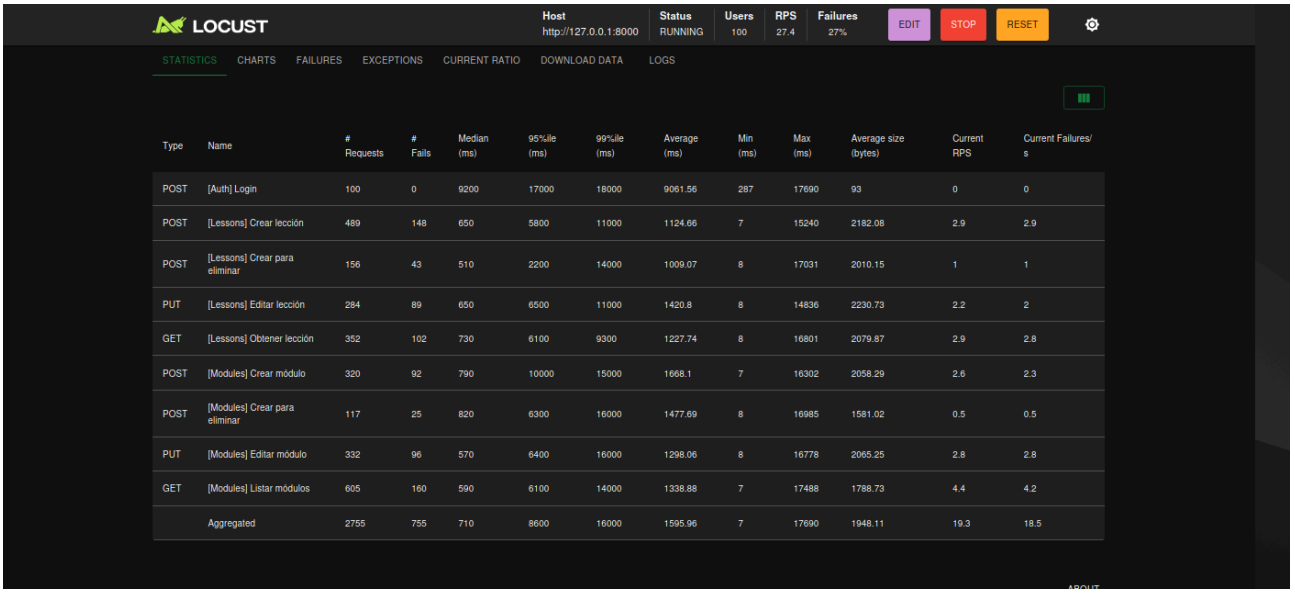
Los tiempos de respuesta bajo carga moderada muestran:

- Promedio general de 134 ms, el mas bajo bajo carga de todos los microservicios
- Percentil 95 en 720 ms
- Las operaciones de lectura mantienen medianas por debajo de 30 ms aun con 30 usuarios
- El login sube a un p95 de 1000 ms bajo 30 usuarios concurrentes

Estos resultados confirman la excelente escalabilidad del Content Service, siendo el microservicio con mejor rendimiento bajo carga moderada de todo el sistema.

- **Prueba de estres:** En la prueba de estres con 100 usuarios concurrentes, el microservicio de Content fue el unico del sistema que presento fallos, registrando una tasa de error del 27% al momento de captura y llegando al 39% en algunos momentos de la prueba





Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	100	0	9200	17000	18000	9061.56	287	17690	93	0	0
POST	[Lessons] Crear lección	489	148	650	5800	11000	1124.66	7	15240	2182.08	2.9	2.9
POST	[Lessons] Crear para eliminar	156	43	510	2200	14000	1009.07	8	17031	2010.15	1	1
PUT	[Lessons] Editar lección	284	89	650	6500	11000	1420.8	8	14836	2230.73	2.2	2
GET	[Lessons] Obtener lección	352	102	730	6100	9300	1227.74	8	16801	2079.87	2.9	2.8
POST	[Modules] Crear módulo	320	92	790	10000	15000	1668.1	7	16302	2058.29	2.6	2.3
POST	[Modules] Crear para eliminar	117	25	820	6300	16000	1477.69	8	16985	1581.02	0.5	0.5
PUT	[Modules] Editar módulo	332	96	570	6400	16000	1298.06	8	16778	2065.25	2.8	2.8
GET	[Modules] Listar módulos	605	160	590	6100	14000	1338.88	7	17488	1788.73	4.4	4.2
	Aggregated	2755	755	710	8600	16000	1595.96	7	17690	1948.11	19.3	18.5

El sistema alcanzo aproximadamente 27-29 RPS antes de comenzar a fallar, siendo el RPS mas alto del sistema. Sin embargo, la gráfica muestra como la linea de fallos crece progresivamente a

partir de los 100 usuarios, lo que indica que Express/Node.js comienza a rechazar peticiones por limites en las conexiones concurrentes con MongoDB.

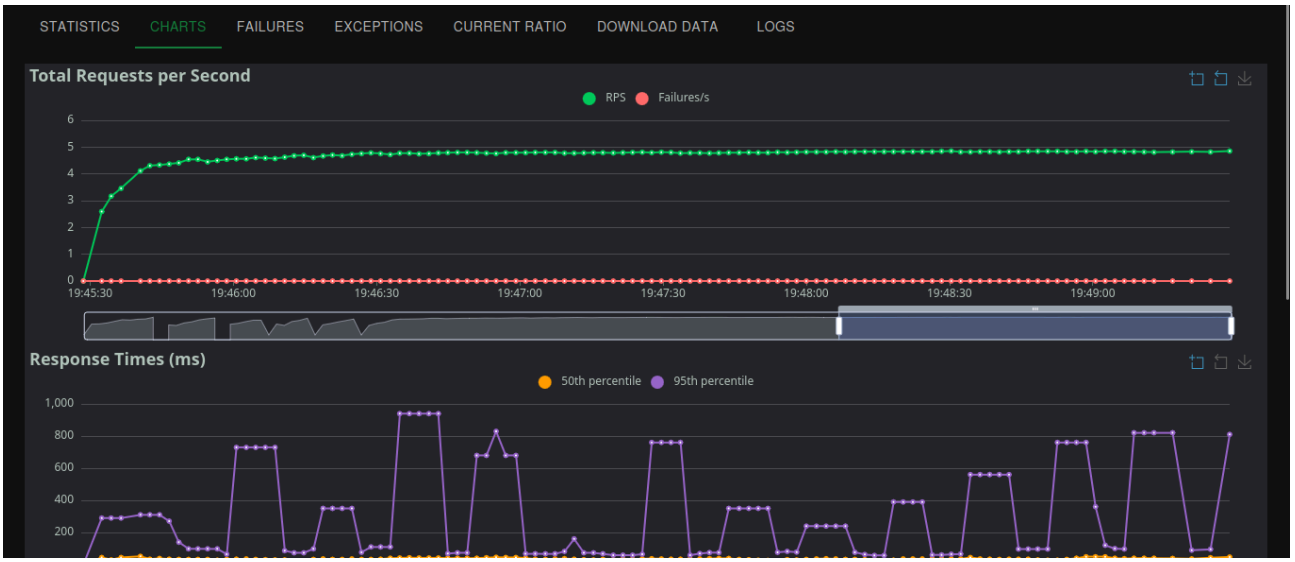
Los resultados bajo estres muestran:

- Tasa de error del 27-39%, la unica significativa en todo el sistema
- Mediana general de 710 ms, relativamente baja a pesar de los fallos
- Percentil 95 de 8,600 ms con picos de hasta 17,690 ms
- Los endpoints mas afectados son Listar modulos (160 fallos) y Crear leccion (148 fallos)

Esto identifica al microservicio de Content como el punto mas critico del sistema bajo alta concurrencia.

10.4 Resultados – Microservicio Evaluations

- **Prueba de capacidad:** En este escenario se evaluo el microservicio de Evaluations con 10 usuarios concurrentes, cubriendo operaciones sobre examenes, preguntas y submissions en condiciones normales.



STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	10	0	290	370	370	292.06	250	372	93	0	0
POST	[Auth] Logout	10	0	160	380	380	193.39	24	385	24	0	0
POST	[Exams] Crear	89	0	41	110	960	83.58	21	962	55.64	0.2	0
POST	[Exams] Crear para eliminar	56	0	37	760	940	95.96	21	937	55.54	0.3	0
PUT	[Exams] Editar	95	0	35	300	920	69.27	20	916	47	0.5	0
GET	[Exams] Listar	130	0	26	270	840	57.37	12	895	6850.08	0.1	0
GET	[Exams] Obtener	79	0	27	390	970	66.8	12	968	3274.62	0.7	0
POST	[Questions] Crear	111	0	40	290	870	81.48	21	967	152.49	0.6	0
POST	[Questions] Crear para eliminar	43	0	38	180	730	70.44	13	728	96.7	0	0
PUT	[Questions] Editar	86	0	23	150	550	39.22	10	551	33	0.4	0
GET	[Questions] Obtener	87	0	25	170	780	52.07	10	781	26	0.4	0
POST	[Submissions] Crear	81	0	45	85	150	51.54	27	148	72.44	0.2	0
POST	[Submissions] Crear para eliminar	38	0	52	990	990	103.1	32	994	73	0.3	0
PUT	[Submissions] Editar	74	0	45	560	750	100.96	17	751	33	0.4	0
GET	[Submissions] Listar	84	0	32	420	810	67.54	12	809	5992.96	0.5	0
GET	[Submissions] Obtener	76	0	27	70	740	36.05	7	736	0	0.2	0
	Aggregated	1149	0	35	290	870	70.44	7	994	1482.66	4.8	0

Los resultados muestran un RPS estable de 4.8 solicitudes por segundo, sin presencia de errores (0% de fallos). La gráfica de RPS muestra una curva ascendente muy limpia que se estabiliza rapidamente, con tiempos de respuesta que se mantienen consistentemente bajos durante toda la prueba.

En cuanto a los tiempos de respuesta, se observa que:

- El tiempo promedio general es de 70.44 ms con una mediana de 35 ms
- El percentil 95 general alcanza 290 ms
- Las operaciones de lectura (listar exams, obtener questions) presentan medianas de 26 ms y 25 ms respectivamente
- Las submissions muestran tiempos muy eficientes con mediana de 45 ms para crear y 32 ms para listar

El sistema procesa 1,149 peticiones en total sin ningun fallo, confirmando el correcto funcionamiento del microservicio de evaluaciones en condiciones normales.

- **Prueba de carga:** Para este escenario se incremento el numero de usuarios concurrentes a 30, evaluando el comportamiento del microservicio de Evaluations bajo demanda moderada.

STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	30	0	780	1300	1500	812.41	248	1485	93	0	0
POST	[Exams] Crear	261	0	42	820	1000	177.13	20	1259	56	1.3	0
POST	[Exams] Crear para eliminar	137	0	40	820	910	142.69	19	953	56	0.5	0
PUT	[Exams] Editar	280	0	37	850	970	144.81	19	981	47	0.8	0
GET	[Exams] Listar	377	0	39	760	930	139.38	15	970	24574.6	1.3	0
GET	[Exams] Obtener	275	0	34	800	960	143.6	14	1153	13092.29	0.9	0
POST	[Questions] Crear	409	0	46	720	930	137.83	23	1002	153	1.9	0
POST	[Questions] Crear	416	0	46	720	930	136.7	23	1002	153	2.3	0
POST	[Questions] Crear para eliminar	118	0	49	890	930	207.01	12	991	88.69	0.6	0
PUT	[Questions] Editar	278	0	25	860	950	153.24	10	1095	33	1.1	0
GET	[Questions] Obtener	253	0	24	870	980	144.34	10	1422	26	1.2	0
POST	[Submissions] Crear	239	0	59	930	1100	184.54	31	1377	72.43	0.8	0
POST	[Submissions] Crear para eliminar	142	0	62	870	960	170.54	32	997	73	0.4	0
PUT	[Submissions] Editar	274	0	38	770	940	144.79	16	993	33	0.7	0
GET	[Submissions] Listar	252	0	35	780	970	141.28	15	986	20801.4	0.8	0
GET	[Submissions] Obtener	264	0	15	700	940	127.46	8	1033	0	0.8	0
	Aggregated	3627	0	41	830	970	155.02	8	1485	5196.71	13.9	0

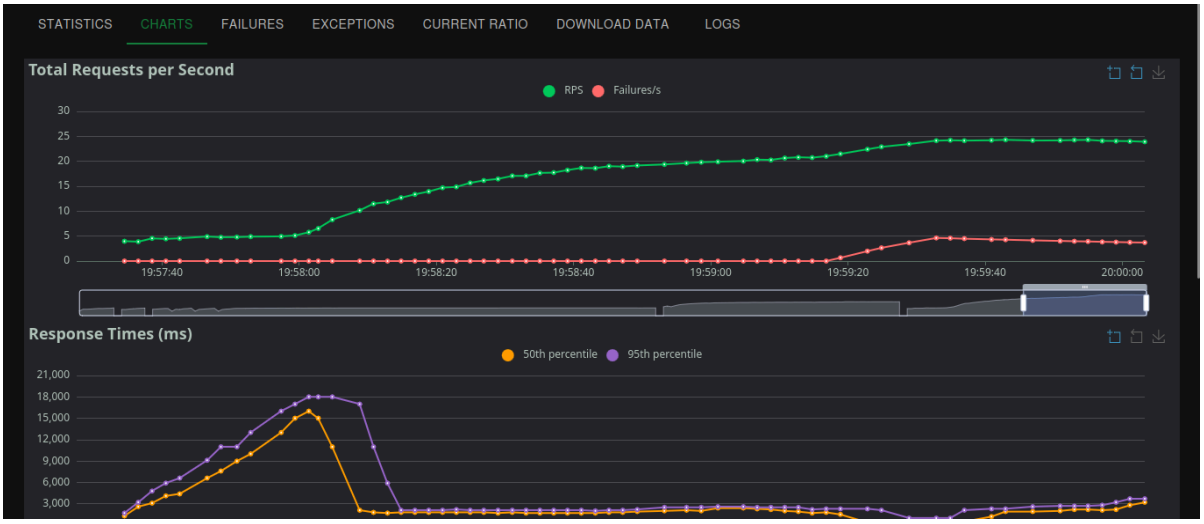
El sistema logro mantener un RPS de 13.91 solicitudes por segundo sin errores registrados. La gráfica muestra una curva de RPS muy estable con poca variacion, evidenciando una excelente gestion de la concurrencia. Los tiempos de respuesta muestran un pico inicial que desciende y se mantiene por debajo de 1,000 ms en p95.

Los tiempos de respuesta bajo carga moderada presentan:

- Promedio general de aproximadamente 150 ms
- Percentil 95 alrededor de 850 ms
- El login sube a un p95 de 1,300 ms, siendo el endpoint mas impactado
- Las operaciones de exams y questions mantienen medianas por debajo de 50 ms

Estos resultados confirman que el microservicio de Evaluaciones escala adecuadamente entre 10 y 30 usuarios concurrentes sin degradacion significativa en la tasa de errores.

- **Prueba de estres:** En la prueba de estres con 100 usuarios concurrentes, el microservicio de Evaluations presento fallos en operaciones de escritura, registrando una tasa de error general del 17%.



LOCUST

Host	Status	Users	RPS	Failures
http://127.0.0.1:8000	RUNNING	100	24.16	17%

EDIT STOP RESET

STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	100	0	9000	17000	18000	9494.8	296	18421	93	0	0
POST	[Exams] Crear	240	59	1700	11000	15000	2083.6	7	16070	1669.4	1.3	0
POST	[Exams] Crear para eliminar	141	30	1700	2500	15000	1698.77	8	15642	1452.38	1.3	0
PUT	[Exams] Editar	245	60	1700	2500	16000	1664.47	8	16852	1656.47	2	0
GET	[Exams] Listar	361	72	1700	2500	15000	1825.51	8	17923	40154.58	3.9	0
GET	[Exams] Obtener	242	54	1700	2500	9500	1605.53	7	12843	22197.8	2.1	0
POST	[Questions] Crear	371	79	1700	2700	13000	1743.39	7	16787	1529.92	3	0
POST	[Questions] Crear para eliminar	134	24	1800	2600	8900	1820	7	15059	1264.03	0.9	0
PUT	[Questions] Editar	270	50	1800	2600	15000	1763.83	7	16203	1252.63	2.1	0
GET	[Questions] Obtener	225	47	1800	5300	14000	1925.92	8	17086	1403.2	1.9	0
POST	[Submissions] Crear	228	0	1800	2600	13000	1913.56	42	17544	72.47	1.3	0
POST	[Submissions] Crear para eliminar	133	0	1800	11000	18000	2191.65	37	18330	73	0.7	0
PUT	[Submissions] Editar	229	0	1700	2500	13000	1642.21	18	16001	33	2	0
GET	[Submissions] Listar	224	36	1800	4300	14000	1945.7	7	16285	38129.25	1.8	0
GET	[Submissions] Obtener	241	47	1700	6400	16000	1938.32	7	16191	1290.84	1.1	0
	Aggregated	3459	558	1800	6800	16000	2056.44	7	18421	9522.38	23.7	0

ABOUT

En la prueba de estres con 100 usuarios concurrentes, el microservicio de Evaluations presento fallos en operaciones de escritura, registrando una tasa de error general del 17%.

Se observa que el sistema alcanza aproximadamente 24 RPS antes de comenzar a fallar. La gráfica muestra el patron tipico de saturacion: un pico inicial de tiempos muy altos (~18,000-20,000 ms) durante el spawn masivo, seguido de una estabilizacion progresiva alrededor de los 1,000-3,000 ms en p95 una vez el sistema se adapta.

Los resultados bajo estres muestran:

- Tasa de error del 17%, concentrada principalmente en operaciones de escritura (Crear, Editar)
- El login registra un promedio de 9,494 ms con p95 de 17,000 ms
- Las operaciones de lectura presentan medianas de 1,700 ms bajo estres extremo
- Los endpoints con mas fallos son Exams Crear (59 fallos) y Exams Editar (60 fallos)

Esto indica que bajo alta concurrencia, las operaciones de escritura en MySQL desde Flask comienzan a presentar timeouts. En produccion, esto se mitigaria con un pool de conexiones adecuado y cache para operaciones de lectura frecuentes.

11. Guía de Despliegue Completo

Requisitos previos: PHP 8.2+, Composer, Python 3.12, Node.js 18+, MySQL, PostgreSQL, MongoDB

Paso 1 — Clonar el repositorio

```
git clone https://github.com/tu-usuario/academix.git
cd academix
```

Paso 2 — Configurar bases de datos

```
# MySQL (Gateway + Cursos + Evaluaciones)
mysql -u root -p -e "CREATE DATABASE academix_gateway;"
mysql -u root -p -e "CREATE DATABASE academix_courses;"
mysql -u root -p -e "CREATE DATABASE academix_evaluations;"

# PostgreSQL (Inscripciones)
psql -U postgres -c "CREATE DATABASE academix_enrollments;"

# MongoDB (Contenido — se crea automáticamente al primer insert)
mongod --dbpath /data/db
```

Paso 3 — Levantar cada servicio

Gateway (Laravel)

```
cd api_gateway
composer install
cp .env.example .env && php artisan key:generate
php artisan migrate
php artisan serve --port=8000
```

Courses Service (Flask)

```
cd courses_service && python -m venv venv && source venv/bin/activate
pip install -r requirements.txt && flask db upgrade
flask run --port=5000
```

Content Service (Express)

```
cd content_service && npm install
node server.js
```

Enrollments Service (Django)

```
cd enrollments_service && python -m venv venv && source venv/bin/activate
pip install -r requirements.txt && python manage.py migrate
python manage.py runserver 8080
```

Evaluations Service (Flask)

```
cd evaluations_service && python -m venv venv && source venv/bin/activate
pip install -r requirements.txt && flask db upgrade
flask run --port=5001
```

Paso 4 — Verificar que todos los servicios estén activos

Método	URL	Servicio
GET	`http://localhost:8000/`	Gateway (Laravel)
GET	`http://localhost:5000/api/courses`	Courses Service (Flask)
GET	`http://localhost:3000/api/modules`	Content Service (Express)
GET	`http://localhost:8080/api/enrollments/`	Enrollments Service (Django)
GET	`http://localhost:5001/api/exams`	Evaluations Service (Flask)